



Python – Funktionen

Stephan Karrer

Autor: Stephan Karrer

Funktionen

```
##### Definition
def addiere(x, y):
    return x + y
    x=1 # dieser Code wird nicht erreicht

def OhneRueckgabe(s):
    print(type(s))

##### Aufruf
result = addiere(5, 6)
print(result)          # 11
result = OhneRueckgabe("Hallo")
print(result)          # None
```

- Funktionen machen Berechnungen mit unterschiedlichen Parametern wiederverwendbar
- Funktionen werden mit dem Schlüsselwort `def` definiert
- Rückgabe erfolgt via `return`, dient auch zum Beenden
- Keine Rückgabe liefert `None` als Ergebnis.

Default-Werte für Parameter

```
##### Definition
def addiere(x , y = 1, c = 0):
    return x + y + c

##### Aufruf
result = addiere(5, 6)
print(result)          # 11
result = addiere(5, 1 )
print(result)          # 6
result = addiere(5, 1 , 1)
print(result)          # 7
result = addiere(5, c = 2)
print(result)          # 8
```

- Optionale Parameter können mit Default-Werten angegeben werden
- Optionale Parameter müssen nach den obligatorischen Parametern stehen
- Parameter können auch mittels ihres Bezeichners angegeben werden (keyword arguments), Reihenfolge bestimmt der Aufrufer
 - Aber dadurch namenstechnische Kopplung !

Variable Parameterliste

```
def f(a, b, *weitere):
    print("Feste Parameter:", a, b)
    print("Weitere Parameter:", weitere)
    print(type(weitere))          # <class 'tuple'>
    s = a+b
    for p in weitere:
        s += p
    return s

##### Aufruf
f(1, 2)
result = f(1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
print(result) # 55
```

- Pflicht-Parameter müssen zuerst angegeben werden
- Die zusätzlichen Parameter stehen als Tupel zur Verfügung

Variable Parameterliste mit Key-Word-Parametern

```
def f(a, b, **weitere):
    print("Feste Parameter:", a, b)
    print("Weitere Parameter:", weitere)
    print(type(weitere))          # <class 'dict'>

def g(*t, **d):
    print(type(t))                # <class 'tuple'>
    print(type(d))                # <class 'dict'>

##### Aufruf
f(1, 2)
f(1, 2, x=3, y=4)
g(1, 2, x=3, y=4)
```

- Pflicht-Parameter müssen zuerst angegeben werden
- Die zusätzlichen Parameter stehen als Dictionary zur Verfügung

Reine Key-Word-Parameter

```
# beliebig viele Positionsparameter vor den Key-Word-Parametern
def f(a, b, *args, d, e=5):
    print(a, b, args, d, e)

# beliebig viele Positionsparameter mit mindestens 2 Key-Word-Parametern
def g(a, b, *args, d, e, **kwargs):
    print(a, b, args, d, e, kwargs)

# Positionsparameter vor den Key-Word-Parametern
def h(a, b, *, d, e):
    print(a, b, d, e)

#### Aufruf
f(1, 2, 3, d=4)                # 1 2 (3,) 4 5
g(1, 2, 3, d=4, e=5, x=20)     # 1 2 (3,) 4 5 {'x': 20}
h(1, 2, d=4, e=5)             # 1 2 4 5
```

- Die positionalen Parameter werden durch * von den Key-Word-Parametern getrennt und beide Formen müssen entsprechend angegeben werden

Reine Positions-Parameter

```
# Positionsparameter vor den Key-Word-Parametern
def f(a, b, /, c, d):
    print(a, b, c, d)

# nur Positionsparameter zulässig
def g(a, b, /):
    print(a, b)

#### Aufruf
f(1, 2, c=3, d=4)          # 1 2 3 4
g(1, 2)                    # 1 2
```

- Die positionalen Parameter werden durch / von den Key-Word-Parametern getrennt und beide müssen entsprechend angegeben werden

Vorsicht: Seiteneffekte bei Default-Parametern

```
def f(a = []):  
    a += [1]  
    print(a)  
  
def g(a = None):          # besser  
    if a is None:  
        a = []  
        a += [1]  
    print(a)  
  
#### Aufruf  
f()          # [1]  
f()          # [1, 1]  
f()          # [1, 1, 1]  
g()          # [1]  
g()          # [1]
```

- Der Default-Parameter wird als Wert (Objekt) für veränderbare Datentypen nur einmal im Speicher erzeugt !